

دليل تشغيل — تطبيقات Fly GACA على iOS (العائلة الأصلية + حضور المتجر)

آخر تحديث 10-08-2026

المالك Founder

active

الحالة

النوع runbook

القسم operations-it-06

المسار الحالي (أغسطس 2026). استراتيجية iOS التي سَنُطلق هي عائلة التطبيقات الأصلية — تطبيق SwiftUI واحد يعمل دون اتصال لكل وحدة دراسية (ELPT و AIP؛ أما وحدات اختبارات الرخص PPL و CPL و IR و ATPL فمعلّقة منذ 10-08-2026 وأزيلت أهدافها)، بدفع مسبق لمرة واحدة، يُبنى من المستودع المشترك `ay2m/FlyGACA` (حزمة FlyGACAKit + لقطات محتوى لكل تطبيق تُزامن من مستودع الويب الرئيسي). أما غلاف Capacitor + اشتراك Pro أحادي التطبيق الذي وثّقه هذا الدليل أصلاً فقد استُبدل لأغراض الإطلاق، ويبقى أدناه مرجعاً لمرحلة اشتراكات مستقبلية.

0. عائلة التطبيقات الأصلية — مسار الإصدار

ما الذي يُشحن: ستة تطبيقات مستقلة على App Store (معزّفات الحزم `com.flygaca.<module>`)، بمجموعة الميزات نفسها دون اتصال (وضع الدراسة، الاختبارات القصيرة، البطاقات التعليمية بالتكرار المتباعد، الاختبارات التجريبية، ومحاكاة اختبار موقوتة ومصححة) — بلا حسابات، بلا مشتريات داخل التطبيق، بلا شبكة في الإصدار الأول؛ يُحزم المحتوى وقت البناء. الموجة 1 = AIP · ELPT · PPL؛ الموجة 2 = ATPL · IR · CPL (محتوى بنوكها قيد المراجعة). التفاصيل التشغيلية تعيش مع الكود — وهذا القسم يُحيل فقط:

1. مزامنة المحتوى — `scripts/sync-content.sh` في `ay2m/FlyGACA`؛ يبقى المستودع الرئيسي مصدر الحقيقة للمتن (آخر مزامنة كاملة 05-08-2026).

2. البناء / الاختبار / CI — في مستودع العائلة: `docs/RUNBOOK-ios-release.md` و `github/workflows/ios.yml`؛ اختبارات Swift، والتحقق من XcodeGen، ومصفوفة بناء لكل تطبيق، ومسار TestFlight يتفعل متى وُجدت أسرار التوقيع.

3. بوابة Apple (عمل بشري لمرة واحدة) — `docs/RUNBOOK-ios-signing.md` + `docs/RUNBOOK-ios-signing-CHECKLIST.md`؛ مجموعة التطبيقات `group.com.flygaca.study`؛ معزّفات التطبيقات (أزيل تسجيل الدخول عبر Apple من ملف الاستحقاقات المشترك في أغسطس 2026، فلا حاجة إلى تجميع الصلاحية؛ وإن عاد فالمعرّف الأساسي هو `com.flygaca.elpt`)؛ شهادة التوزيع؛ ملفات التزويد `FlyGACA <APP> AppStore`؛ سجلات App Store Connect المدفوعة؛ مفتاح ASC API؛ ثم أسرار GitHub العشرة (`scripts/native/set-signing-secrets.sh`).

4. Firebase (للمرحلة المتصلة لاحقاً) — `docs/RUNBOOK-ios-firebase.md`؛ أمر `npm run firebase:register` مع الخطوتين اليدويتين: مزوّد تسجيل الدخول عبر Apple ومفتاح APNs. لا ضرر من إنجازه مبكراً؛ وليس مطلوباً للإصدار الأول دون اتصال.

5. قوائم المتجر — مستودعات البيانات الوصفية الستة `FlyGACA/<MODULE>`؛ نصوص إنجليزية وعربية ولقطات شاشة، يحرسها فحص CI لكل مستودع (الحدود + تكافؤ اللغتين). نص القوائم يصف بصدق تطبيقاً يعمل دون

قائمة مراجعة Apple — الإصدار الأول دون اتصال

- لا مشتريات داخل التطبيق ولا روابط شراء خارجية — التطبيقات مدفوعة مسبقاً، فالبند 3.1.1 لا ينطبق.
- لا نظام حسابات في الإصدار الأول — يصبح تسجيل الدخول عبر Apple (4.8) وحذف الحساب/البيانات داخل التطبيق (5.1.1) مطلبين فقط عندما تُشحن مرحلة الاتصال (PlatformLive) بتسجيل الدخول. وتظل *الخاصية* على معرّفات التطبيقات مطلوبة للتوقيع لأن ملف الاستحقاقات المشترك يعلنها.
- ملصقات تغذية الخصوصية: «لا تُجمع بيانات» (دون اتصال؛ لا تحليلات ولا تتبع). يعاد النظر فيها قبل إضافة أي قياس عن بُعد مستقبلاً.
- يبقى تنويه «ليس للاستخدام التشغيلي» بارزاً في كل تطبيق وكل قائمة — استخدام تعليمي فقط؛ GACA هي الجهة المرجعية دائماً.
- البند 4.3(ب) (التطبيقات المكررة) — ستة تطبيقات شقيقة من قالب واحد: جهّز ملف التمايز وفق `SEO-PLAN.md` 3.1§ في مستودع العائلة قبل التقديم.

المسار المستبدل — تطبيق Capacitor واحد + اشتراك Pro (خيار مستقبلي)

كل ما يلي يؤثّق خطة ما قبل أغسطس 2026: غلاف Capacitor واحد `com.flygaca.app` حول تطبيق الويب PWA، مع تحقيق الدخل عبر RevenueCat / مشتريات Apple. الكود المشار إليه في المستودع الرئيسي ما يزال موجوداً، ويبقى هذا مرجعاً إن أُحْيى غلاف الاشتراكات — لكنه ليس مسار إطلاق App Store الحالي.

يغلّف تطبيق iOS تطبيق الويب التقدمي القائم (PWA) داخل غلاف Capacitor ويبيع Fly GACA Pro عبر مشتريات Apple داخل التطبيق. ويعيد استخدام كامل الخلفية: فالشراء يمنح `users/{uid}.entitlement` (تكتبه Cloud Function وحدها)، ويقرأه التطبيق عبر مسار `store.js` / `entitlements.js` نفسه الذي يستخدمه الويب.

ما هو موجود بالفعل في المستودع (مبني وآمن على الويب):

- `capacitor.config.ts` — `appld` `com.flygaca.app` ، `webDir: www`.
- `npm run build:ios` (`scripts/build-ios-web.js`) — يجعّ `www/` : يحزم `parts/` الخاصة بـ GACAR، و `charts/` الخاصة بـ VFR، والفهارس، والأدوات، والدراسة، والأدلة الإرشادية (~26 ميجابايت)؛ ويستثني أدلة FAA/الأجنبية، والكتب الإلكترونية، وفهرس البحث البالغ 19 ميجابايت (تُبثّ + تُخزّن على الجهاز). ويحقن `assets/js/native-bridge.js`.
- `assets/js/native-bridge.js` — رقعة أصلية فقط: توجّه دعوة جدار الدفع إلى ورقة StoreKit، وشراء/استعادة RevenueCat، وتخزين جلب مؤقت للأصول المبنوثة. بلا أثر على الويب.
- `assets/js/auth.js` — فرع تسجيل دخول أصلي عبر Apple/Google (لا يزال الويب يستخدم النافذة المنبثقة).
- `assets/js/gate.js` — جدار الدفع حيّ فقط داخل التطبيق (`GATING_LIVE`) يكون `true` فقط عندما يكون `Capacitor.isNativePlatform()`؛ ويبقى الويب مفتوحاً.
- `functions/revenuecatWebhook.js` — RevenueCat ← `users/{uid}.entitlement`

- `firebase.json` — CORS على `/assets/data/**` كي يستطيع التطبيق بثّ الأصول الثقيلة.
- كل ما يلي يحتاج إلى جهاز Mac مع Xcode + CocoaPods وحسابي RevenueCatg Apple Developer — ولا يمكن إنجازه في CI/Linux.

1. توليد المشروع الأصلي (Mac)

```
npm install --legacy-peer-deps      # .npmrc already sets this
npm run build:ios                    # writes www/
npx cap add ios                      # generates ios/ (Xcode project + Pods)
npx cap sync ios                    # or: npm run cap:sync
npx cap open ios                    # opens Xcode
```

في Xcode: عيّن الفريق/التوقيع، ومعرّف الحزمة `com.flygaca.app`، وفعل قدرتي Sign ing In-App Purchase و Push Notificationsg) with Apple عند وصول المرحلة 2).

2. مصادقة Firebase الأصلية

- أضف iOS app إلى مشروع Firebase `flygaca-firebase`؛ نزل `GoogleService-Info.plist` إلى `ios/App/App/`.
- فعل موثري Googleg Apple في Firebase Auth (ففعّلن بالفعل للويب). أضف معرّف حزمة iOS. لتسجيل الدخول عبر Google، أضف مخطط عنوان URL لمعرّف العميل المعكوس إلى هدف iOS.
- تحقق من أشكال طريقة/نتيجة `@capacitor-firebase/authentication` مقابل الإصدار المثبت (v7.x):
`signInWithGoogle()` ، `signInWithApple()` ← `result.credential.{idToken, nonce, accessToken}`
ويبادل الفرع في `auth.js` هذه إلى JS SDK عبر `signInWithCredential`.

3. App Store Connect — منتجات الاشتراك

- أنشئ مجموعة اشتراك ذاتية التجديد "Fly GACA Pro" تتضمن:
- `pro_monthly` — 59 ريالاً سعودياً / شهرياً.
 - `pro_annual` — 349 ريالاً سعودياً / سنوياً، مع تجربة مجانية 7 أيام (عرض تمهيدي). (تتطابق معرّفات المنتجات التي تحتوي `year / annual` مع `source: 'annual'` في الـ `webhook`؛ عدّل `periodOf()` في `functions/revenuecatWebhook.js` إذا اخترت معرّفات أخرى). أضف البيانات الوصفية المدّلية المطلوبة، ولقطة شاشة المراجعة، وروابط شروط الاشتراك / الخصوصية.

4. RevenueCat

- أنشئ مشروعاً، وأضف تطبيق App Store + السر المشترك الخاص بالتطبيق (App-Specific Shared Secret).

- أنشئ Offering بحزمتي `$rc_monthly` و `$rc_annual` مرتبطين بالمنتجين، و Entitlement باسم `pro` مرتبطاً بكليهما.
- انسخ مفتاح SDK العام لـ iOS ← عيّنه للتطبيق، إما مضمّناً في `native-bridge.js` (`REVENUECAT_IOS_KEY`) أو اعرض `window.FG_REVENUECAT_IOS_KEY` قبل تحميل ذلك السكريبت. إنه مفتاح قابل للنشر، آمن للشحن.
- Webhook: وجهه إلى عنوان URL للدالة المنشورة (`https://me-central1-flygaca-` أو `firebase.cloudfunctions.net/revenuecatWebhook` أو عنوان Cloud Run) واضبط ترويسة Authorization على سرّ قوي.
- يستدعي التطبيق `Purchases.logIn(firebaseUid)` بعد تسجيل الدخول (عبر `FGNative.setUser`) كي يساوي `app_user_id` معرفّ Firebase uid.

5. نشر الـ webhook

```
firebase functions:secrets:set REVENUECAT_WEBHOOK_AUTH # same value as the RC header
cd functions && npm ci && cd ..
firebase deploy --only functions:revenuecatWebhook,hosting
```

(يشحن نشر `hosting` ترويسة CORS الجديدة لـ `/assets/data/**`.)

6. التحقق من حلقة الشراء (وضع الحماية في StoreKit)

1. سجّل الدخول (Apple/Google) على جهاز/محاكٍ.
2. الجأ إلى ميزة محجوبة (مثل أداة طيران رابعة، أو الدراسة) ← يظهر جدار الدفع Go Pro ← اشترِ بحساب وضع الحماية.
3. يُظهر RevenueCat استحقاق `pro` نشطاً ← يكتب الـ webhook الخاص به `users/{uid}.entitlement` (تحقق في Firestore) ← يعيد التطبيق التحميل و تُرفع البوابة.
4. سجّل الدخول إلى الويب بالحساب نفسه ← يظهر Pro هناك أيضاً.
5. اختبر Restore Purchases والتجربة التمويديّة السنوية.
6. دون اتصال: افتح Part من GACAR وخريطة مع إيقاف الشبكة (محزومة ← تعمل)؛ افتح دليل FAA على الإنترنت مرة، ثم دون اتصال (مبثوث ← مخزن).

قائمة تحقق مراجعة Apple

مشتريات داخل التطبيق فقط لـ Pro (3.1.1) • تسجيل دخول أصلي عبر Sign in with Apple (4.8) • حساب داخل التطبيق + حذف البيانات (موجود بالفعل في `settings.js` ، 5.1.1) • Restore Purchases • ملصقات تغذية الخصوصية • أبقِ إخلاء المسؤولية "ليس للاستخدام التشغيلي" بارزاً.

المرحلة 2 — تذكيرات انتهاء الصلاحية + حزم التحضير (في المستودع؛ تحتاج إلى ربط بالجهاز)

تذكيرات انتهاء الصلاحية الدفعية

الشفيرة موجودة بالفعل: `functions/reminders.js` (وظيفة مجدولة يومية `expiryReminders` الساعة 06:00 بتوقيت Asia/Riyadh — تفحص `users` ، وتدفع عندما يتجاوز `medicalExpiry` / `lastFlightReview` +24 شهراً عتبة). و `store.js savePushToken` ، و `native-bridge.js registerPush` ، وخطاف لوحة المعلومات الذي يسجل عند تسجيل الدخول، للتفعيل:

1. Apple Developer ← أنشئ (p8) APNs Auth Key: ارفعه إلى Cloud Messaging ← Firebase Project Settings (iOS).

2. Xcode ← أضف قدرة Push Notifications (Remote notifications ← Background Modes) إلى هدف App.

3. `npx cap sync ios` (تبعية `@capacitor/push-notifications` موجودة بالفعل في `package.json`).

4. النشر: `firebase deploy --only functions:expiryReminders`.

5. التحقق: عيّن ملفاً شخصياً `medicalExpiry` بعد 7 أيام (Settings)، وتأكد من وصول رمز إلى `users/{uid}.fcmTokens` ، ثم شغل الدالة "Cloud Scheduler "Run now" أو المحاكى) وتأكد من وصول الإشعار الدفعي.

حزم التحضير لمرة واحدة (غير قابلة للاستهلاك)

الشفيرة موجودة بالفعل: يربط الـ `webhook` `pack_elpt` / `pack_aip` ← `entitlement.packs[]` (ضمن معاملة، مع الحفاظ على أي خطة Pro)؛ `native-bridge.js purchasePack` ؛ ودعوة جدار الدفع توجه ميزات `pack:*` إليه؛ `gate.js` / `entitlements.js hasPack` يفتحان بالفعل `packs/*.html`.

1. App Store Connect ← أنشئ منتجات غير قابلة للاستهلاك `pack_aip` ، `pack_elpt` (وسّع `PACK_PRODUCTS` في `revenuecatWebhook.js` للمزيد).

2. RevenueCat ← أضف المنتجات؛ يعالج الـ `webhook` `NON_RENEWING_PURCHASE`.

3. التحقق: افتح `packs/aip.html` بصفتك غير مالك ← جدار الدفع ← "Unlock this pack" ← شراء بوضع الحماية ← يضيف الـ `webhook` `aip` إلى `packs[]` ← تفتح الصفحة؛ وتأكد من أن تجديد Pro لاحقاً لا يُسقط الحزمة (تحفظ المعاملة كليهما).

المرحلة 3 — فوتر Stripe على الويب + تراخيص مدارس B2B (في المستودع)

تمرّ الآن كل عمليات كتابة الاستحقاقات عبر مساعد معاملاتي واحد (`functions/entitlements.js`)، تتشاركه مسارات Stripe و Apple والمدارس، كي لا يطمس اشتراك ولا الحزم لمرة واحدة ولا مقعد مدرسة أي منها الآخر أبداً.

Stripe على الويب (هامش أعلى من مشتريات Apple داخل التطبيق)

الطيار التجريبي الذي يشترك على الويب يحصل على الاستحقاق نفسه عبر المنصات (Pro في التطبيق أيضاً) دون اقتطاع Apple. يجب ألا يربط التطبيق إطلاقاً أو يذكر دفع الويب (القاعدة 3.1.1) — يرفض `billing.js` بدء الدفع داخل التطبيق. الشيفرة موجودة: `functions/stripe.js` (`createCheckoutSession`) القابلة للاستدعاء +

(stripeWebhook), (assets/js/billing.js) (FGBilling.startProCheckout).

1. Stripe Dashboard ← أنشئ "Product "Fly GACA Pro" بسعرين متكررين: 59 ريالاً سعودياً / شهرياً و349 ريالاً سعودياً / سنوياً.

2. الأسرار: `firebase functions:secrets:set STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET`
`.STRIPE_PRICE_MONTHLY STRIPE_PRICE_ANNUAL`

3. `cd functions && npm i && cd ..` (يضيف حزمة stripe)، ثم `firebase deploy --only`
`.functions:createCheckoutSession,functions:stripeWebhook`

4. Stripe ← أضف webhook endpoint عند عنوان `stripeWebhook` المنشور، مشتركاً في
`checkout.session.completed` , `customer.subscription.created/updated/deleted` . ضع سّر توقيعه
في `STRIPE_WEBHOOK_SECRET` .

5. افعل دعوة الويب (مهمة إطلاق): على `pricing.html` , حقل `assets/js/billing.js` واربط دعوة Pro بـ
`FGBilling.startProCheckout('annual'|'monthly')` . ولجعل جدار دفع الويب يحجب المشتركين غير
المشاركين فعلاً، عيّن `WEB_GATING_LIVE = true` في `assets/js/gate.js` — افعل ذلك فقط بعد أن
يصبح الكيان القانوني + الدفع حثيين.

6. التحقق: مستخدم ويب مسجل الدخول ← الدفع ← بطاقة اختبار ← `checkout.session.completed` ←
`Pro = users/{uid}.entitlement` ← سجل الدخول على iOS، يظهر Pro هناك أيضاً.

تراخيص مدارس B2B (تباع خارج التطبيق)

تشترى المدارس مقاعد بموجب عقد؛ ثم يزود مشغل الطلاب. الشيفرة موجودة: `functions/school.js`
(`grantSchoolLicence` / `revokeSchoolLicence` , للمسؤول فقط).

1. اجعل مشغلاً مسؤولاً (لمرة واحدة): `( admin.auth().setCustomUserClaims(uid, { admin: true })`
(سكربت Node صغير مع حساب الخدمة، أو صدفقة دالة Firebase console).

2. النشر: `firebase deploy --only functions:grantSchoolLicence,functions:revokeSchoolLicence` .

3. التزويد: استدع `grantSchoolLicence({ emails:[...], schoolId, expiresAt })` (مثلاً من أداة مسؤول
أو `firebase functions:shell`). يصبح استحقاق كل طالب `{ plan:'school', source:'school',`
`schoolId, expiresAt }` وهو ما يعامله `isPro()` بالفعل بوصفه وصولاً كاملاً. استخدم
`revokeSchoolLicence` عند نهاية العقد.